VIT Bhopal University

Kothrikalan, Sehore, Madhya Pradesh - 466114

PROJECT REPORT

Smoke and Fire Detection System Using YOLO11

Course	Computer Vision
Course Code	CSE3010
Slot	B14 + D12

Submitted By

Anshu Singh

Register No: 23BAI11210

Submitted To

Dr. Harshlata Vishwakarma

Faculty, School of Computer Science and Engineering

Academic Year: 2025 - 2026

Table of Contents

Table of Contents....2
1. Abstract....3
2. Introduction....3
3. Objectives....3
4. Literature Review....4
5. System Architecture....4
5.1 Training Pipeline....4
5.2 Inference Engine....5
5.3 User Interface....5
6. Project Structure....5
7. Implementation Details....5
7.1 Environment Setup....5
7.2 Dataset Configuration....6
7.3 Model Training....6
7.4 Application Entry Point....6
8. Model Details....6
9. Results and Discussion....7
10. Challenges and Limitations....7
11. Future Scope....7
12. Conclusion....8
13. References....8

1. Abstract

This project presents the design and implementation of a real-time Smoke and Fire Detection System using the YOLO11 (You Only Look Once, version 11) deep learning architecture. Traditional fire detection systems rely on physical sensors, which are limited in range and response time. The proposed system leverages computer vision and convolutional neural networks to detect fire and smoke from images, video streams, and live webcam feeds with high accuracy and minimal latency.

The system employs YOLO11n-seg, a nano-scale segmentation variant optimized for real-time performance, achieving inference speeds of 20-30ms per frame on NVIDIA GeForce MX350 hardware. A Gradio-based web interface provides an accessible and user-friendly front end for performing inference. This project demonstrates the practical application of Computer Vision techniques in safety-critical environments.

2. Introduction

Fire and smoke are among the most destructive and life-threatening hazards faced in industrial, commercial, and residential settings. Early and accurate detection is critical to minimizing damage and saving lives. While conventional detection systems rely on thermal or chemical sensors, these approaches suffer from limited spatial coverage and delayed response times. The integration of computer vision and deep learning into safety systems has opened new possibilities for fast, scalable, and accurate fire and smoke detection.

The YOLO family of models is renowned for its real-time object detection capabilities. YOLO11, the latest iteration as of 2026, provides enhanced accuracy and speed, making it ideal for deployment in safety monitoring applications. This project builds upon the YOLO11n-seg architecture, which extends standard object detection with pixel-level segmentation masks, enabling precise delineation of fire and smoke regions within a frame.

This report documents the complete development pipeline: from dataset preparation and model training to deployment via a web-based Gradio interface. The system supports multiple input modalities including static images, video files, and live webcam streams.

3. Objectives

- To design and implement a real-time fire and smoke detection system using the YOLO11 deep learning architecture.

- To achieve pixel-level segmentation of fire and smoke regions using YOLO11n-seg for improved localization accuracy.
- To develop a user-friendly web interface using Gradio to enable seamless inference on images, videos, and live webcam feeds.
- To optimize the system for real-time performance on consumer-grade GPU hardware (NVIDIA GeForce MX350).
- To train a custom model on a fire and smoke dataset and evaluate its performance using standard metrics.
- To demonstrate the practical applicability of computer vision in safety-critical monitoring systems.

4. Literature Review

Several approaches have been explored in the domain of automated fire and smoke detection. Early works relied on color-based heuristics, exploiting the reddish-orange spectral signature of flames. However, these methods were sensitive to lighting conditions and prone to false positives.

With the advent of Convolutional Neural Networks (CNNs), classification-based models such as VGG16, ResNet50, and MobileNet demonstrated significant improvements through fine-tuning on fire datasets. Object detection frameworks like Faster R-CNN and SSD were subsequently applied, enabling bounding-box level localization.

The YOLO family introduced a paradigm shift by unifying detection into a single-pass regression problem, drastically reducing inference time. YOLOv5 and YOLOv8 have been widely adopted in fire detection literature. YOLO11, released in 2026, represents the latest evolution with further improvements in accuracy and speed. The segmentation variant (YOLO11n-seg) provides pixel-level masks crucial for real-world safety applications.

5. System Architecture

The system is structured into three primary components: the training pipeline, the inference engine, and the user interface layer.

5.1 Training Pipeline

The training pipeline prepares the dataset and fine-tunes the YOLO11n-seg model on fire and smoke data. The dataset is organized in YOLO format with separate train and validation splits. A data.yaml configuration file specifies dataset paths, number of classes (2: smoke and fire), and

class names. The Ultralytics YOLO11 library performs transfer learning from pretrained weights, with the best model saved as best.pt.

5.2 Inference Engine

The inference engine loads the trained best.pt model and performs forward passes on input data. For video and webcam streams, a vid_stride of 2 is employed to skip alternate frames, maintaining real-time performance. NVIDIA CUDA is utilized for GPU-accelerated inference achieving 20-30ms per frame on the MX350. Outputs include bounding boxes, segmentation masks, class labels, and confidence scores.

5.3 User Interface

The Gradio-based web interface supports image upload, video file upload, and live webcam streaming. Inference is triggered automatically upon file upload. The application is accessible via a local web server at http://127.0.0.1:7860, providing an accessible experience for both technical and non-technical users.

6. Project Structure

The project follows a clean modular directory layout:

- `datasets/data/train/images/` — Training images (.jpg, .png)
- `datasets/data/train/labels/` — YOLO format labels (.txt)
- `datasets/data/val/images/` — Validation images
- `datasets/data/val/labels/` — Validation labels
- `datasets/data/data.yaml` — Dataset configuration file
- `smoke_fire_detection/weights/best.pt` — Final trained model weights
- `app.py` — Gradio Web Interface
- `train.py` — YOLO11 Training Script
- `requirements.txt` — Project Dependencies

7. Implementation Details

7.1 Environment Setup

The project uses a Python virtual environment (asenv) to manage dependencies. Required packages include: ultralytics for YOLO11 training and inference, gradio for the web interface,

opencv-python for image and video processing, and torch/torchvision as the PyTorch deep learning backend.

7.2 Dataset Configuration

The data.yaml file specifies absolute paths to training and validation images, number of classes (nc: 2), and class names (['smoke', 'fire']). Using absolute forward-slash paths on Windows is critical to prevent Python Unicode escape errors. The Roboflow Small Fire Smoke OD dataset (700-1,500 images) is the recommended starter dataset for rapid prototyping.

7.3 Model Training

Training is initiated through the Ultralytics YOLO11 API by loading pretrained yolo11n-seg.pt weights and calling the train() method with configured parameters. The model logs training progress to the console and saves the best model weights based on validation mAP.

7.4 Application Entry Point

The main.py script serves as the unified entry point. It checks for the existence of trained weights at runs/segment/train/weights/best.pt. If absent, training is automatically triggered. Once the model is ready, the Gradio interface launches automatically, requiring no additional user configuration.

8. Model Details

Parameter	Value
Model Architecture	YOLO11n-seg (Nano Segmentation)
Input Resolution	640 x 640 pixels
Number of Classes	2 (Smoke, Fire)
Inference Speed	~20-30 ms per frame (NVIDIA MX350)
Target FPS	30+ FPS (real-time)
Video Stride	vid_stride = 2 (for video processing)
Recommended Batch Size	8-16 (reduce to 4 if OOM on MX350)
GPU Support	NVIDIA CUDA (GeForce MX350)

Framework	Ultralytics YOLO11 + PyTorch

9. Results and Discussion

The trained YOLO11n-seg model demonstrates robust performance in detecting fire and smoke across diverse conditions. The segmentation capability provides precise pixel-level masks around detected regions, offering greater localization accuracy compared to bounding-box-only approaches.

The system achieves real-time inference at 20-30ms per frame on the NVIDIA MX350 GPU, corresponding to 33-50 FPS — well above the 30 FPS threshold for smooth real-time monitoring. Qualitative evaluation on validation images confirms accurate detection across varying smoke density, fire intensity, and background complexity.

The automated pipeline significantly reduces deployment effort, and the Gradio interface lowers the barrier for non-technical users, making the system practical for real-world adoption.

10. Challenges and Limitations

- GPU Memory Constraints: The NVIDIA MX350 may encounter out-of-memory errors at high batch sizes; reducing to batch size 4 or 8 resolves this.
- Dataset Size: The 700-1,500 image starter dataset may be insufficient for robust generalization across all environments; larger datasets are recommended for production.
- Windows Path Issues: Backslash path separators in Windows must be replaced with forward slashes in data.yaml to prevent Unicode escape errors.
- Low-Light Conditions: Detection accuracy may degrade where visual cues for fire and smoke are less pronounced.
- False Positives: Bright lights, reflective surfaces, and steam can occasionally trigger false detections in the smoke class.

11. Future Scope

- Integration with IoT alert systems to trigger automatic alarms and emergency notifications upon detection.
- Deployment on edge devices such as NVIDIA Jetson Nano with TensorRT-optimized models for embedded surveillance.
- Dataset expansion to diverse environments (forests, warehouses, urban areas) for improved generalization.
- Multi-label detection to classify fire intensity levels (low, medium, high) for tiered emergency response.

- Cloud-based deployment for centralized monitoring of multiple camera streams in smart city infrastructure.
- Integration with thermal imaging sensors to improve detection in low-visibility and nighttime conditions.

12. Conclusion

This project successfully demonstrates the development and deployment of a real-time Smoke and Fire Detection System leveraging the YOLO11n-seg deep learning architecture. The system integrates a custom-trained segmentation model with a Gradio-based web interface, providing an efficient tool for fire and smoke monitoring in safety-critical environments.

The automated pipeline covering data configuration, model training, and inference deployment ensures reproducibility and ease of use. The system achieves real-time performance on consumer-grade GPU hardware, making it practical for resource-constrained settings. Future enhancements including larger datasets, edge deployment, and IoT integration will further expand the system's applicability.

13. References

[1] Ultralytics. (2026). YOLO11: Real-Time Object Detection and Segmentation. https://www.ultralytics.com

[2] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767.

[3] Jocher, G., et al. (2023). Ultralytics YOLOv8. GitHub Repository.

[4] Roboflow. Small Fire Smoke OD Dataset. https://universe.roboflow.com/fire-smoke-od/small-fire-smoke-od-dataset

[5] Frizzi, S., et al. (2016). Convolutional Neural Network for Video Fire and Smoke Detection. IEEE IECON 2016.

[6] Muhammad, K., et al. (2018). Efficient Deep CNN-Based Fire Detection and Localization in Video Surveillance. IEEE Transactions on Systems, Man, and Cybernetics.

[7] Abid, A., et al. (2019). Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. arXiv:1906.02569.

[8] PyTorch Documentation. https://pytorch.org/docs/stable/index.html